

PRACTICEPRO — CODEBASE AUDIT

Van Clief ICM Methodology Audit

Applying Jake Van Clief's Interpretable Context Methodology to ALOA / ARIA, with a ship-blocker / polish / nice-to-have launch lens

A focused audit of the ALOA / ARIA AI prompt architecture through Jake Van Clief's ICM lens (folder structure as agent architecture, prompts as versioned markdown files, no heavy framework code) — plus a "you're automating the wrong layer" pass over the frontend / backend boundary, and an explicit alpha-launch posture tagging every finding as SHIP-BLOCKER, POLISH, or NICE-TO-HAVE. Genuine engineering findings from the previous audit (multi-tenant auth holes, dead shadcn code, god components, XSS hardening) are preserved and reframed where applicable.

Metric	Value	Status
ICM scaffold markdown files	5 in /ai/prompts/ (numbered)	SHOWCASE
AI surfaces audited (ICM scope)	4 — ALOA/ARIA chat, DraftPro, Research Studio, Proactive Engine	SHOWCASE
Total prompt strings catalogued	33 across the 4 surfaces (5 with MD, 28 code-only)	POLISH
ICM completeness score	~18% — markdown is documentation only, zero build-time loading	POLISH
Identity guardrail gaps found	3 — Research Studio (none), Proactive briefing (minimal), split-brain validator	SHIP-BLOCKER
New ship-blockers found in extended audit	3 — Research Studio unguarded, ARIA expansion drift, briefing identity gap	SHIP-BLOCKER
Van Clief showcases (already correct)	3 — sales-lead pipeline, billing state machine, notifyFounders helper	SHOWCASE
Wrong-layer findings	16 catalogued (3 ship-blockers, 12 polish, 5 nice-to-have)	POLISH
TypeScript errors (post-fix)	315 (was 327; 5 surgical fixes applied in prior session)	SHOWCASE
Build status	Passing (22s)	SHOWCASE
Atrium launch posture	3 ship-blockers must fix; rest is polish / nice-to-have	SHIP-BLOCKER

Report date: 16 August 2026 | Methodology: Jake Van Clief — ICM, "automating the wrong layer", systems thinking | Auditor: Super Z (Z.ai)

Contents

1. The Van Clief Methodology — In Practice	5
1.1 The Four Ideas Applied Here	5
1.2 What This Audit Is NOT	5
1.3 What Is Preserved From the Previous Audit	5
2. ICM Audit — ALOA / ARIA Prompt Architecture	6
2.1 ICM Completeness Score: ~18%	7
2.2 Drift Report — Markdown vs Code	7
2.3 Code-Only Prompts (No Markdown Counterpart)	8
2.4 Identity Guardrail Split-Brain	8
2.5 A Hidden Bug — Atrium Chat Gets the Legal Precision Protocol	9
2.6 Build-Time Loading — Recommended Approach	9
3. ICM Audit — DraftPro Editor	10
3.1 The 3-Variant Split — Why One MD File Is Not Enough	10
3.2 DraftPro Prompt Inventory — 18 Surfaces	10
3.3 The rewriteText Variant-Selection Bug	11
3.4 DraftPro Does NOT Use Function-Calling Tools	12
3.5 DraftPro Migration Priority Order	12
4. ICM Audit — Research Studio	12
4.1 CRITICAL — Research Studio Runs Without Identity Protection	13
4.2 The Wrong ARIA Expansion Bug	13
4.3 Research Studio Prompt Inventory — 12 Surfaces	14
4.4 Research Studio Does NOT Use Function-Calling Tools	14
4.5 Pre-Migration Code Fixes (BLOCKER)	14
5. ICM Audit — Proactive Intelligence Engine	15
5.1 Proactive Engine Prompt Inventory — 3 AI Surfaces	15
5.2 CRITICAL — Morning Briefing Runs Without ARIA Identity Lock	16
5.3 The Convex Runtime Loading Problem	16
5.4 Error Handling — Silent Failures Across the Board	17
5.5 Proposed Markdown Files + Migration Priority	17
6. "You're Automating the Wrong Layer" Audit	17
6.1 The Three Van Clief Showcases (Already Correct)	18
6.2 Wrong-Layer Findings — Header.tsx Notification Bell	18

6.3 Wrong-Layer Findings — DataProvider.tsx Optimistic Updates	19
6.4 Wrong-Layer Findings — Defensive Polling in Founder App	20
6.5 Wrong-Layer Findings — Sequential Per-Message Mutations	20
7. Alpha-Launch Posture — Ship Atrium Now	21
7.1 SHIP-BLOCKERS (Must Fix Before Atrium Launch)	21
7.2 POLISH (Fix in First 2 Weeks Post-Launch)	21
7.3 NICE-TO-HAVE (Backlog)	23
8. Implementation Roadmap	23
8.1 Phase 0 — Ship-Blockers (3-4 days)	23
8.2 Phase 1 — ICM Proof-of-Concept + Quick Wins (1 week)	24
8.3 Phase 2 — Security + Data Integrity (1-2 weeks)	24
8.4 Phase 3 — ICM Extension + Code Organization (2-4 weeks)	25
9. Appendix — Recipes & Reference	25
9.1 The ICM Loader Recipe (Vite ?raw imports)	25
9.2 The notifyFounders Recipe (Already a Showcase)	26
9.3 The ErrorBoundary Recipe (Replaces Defensive Polling)	27
9.4 Files Modified in This Audit Session	27
9.5 Audit Methodology	28

1. The Van Clief Methodology — In Practice

Jake Van Clief (@JEVanClief) teaches a small set of related ideas that share a single posture: stop building complex orchestration code; structure the context instead. The context IS the orchestration. For PracticePro — a multi-product SaaS (Vega / Atrium / Komplete) with an AI copilot (ALOA for legal, ARIA for property) — this audit applies four of his ideas to the codebase and reframes the genuine engineering findings from the prior session through that lens.

1.1 The Four Ideas Applied Here

Idea	What Van Clief means	How this audit applies it
1. ICM (Interpretable Context Methodology)	Folder structure IS the architecture. Numbered folders represent workflow stages. Markdown files carry prompts and context. No heavy framework code. AI workflows become interpretable, maintainable, and editable without deploys.	Audit the existing /ai/prompts/ scaffold (5 numbered MD files). Measure completeness: do code paths actually LOAD the markdown, or is it just documentation? Catalog every prompt string still living in code.
2. You're Automating the Wrong Layer	Most people build complex orchestration code when they should be structuring the context that feeds the system. The notification chain, the badge counting, the debounce logic — these are all "the wrong layer" if they're in the frontend when they could be a single backend event.	Find every place the React client is doing work that should be a Convex mutation or query. Trace the sales-lead, notification, and billing pipelines. Catalog the wrong-layer patterns.
3. Systems Thinking	Treat the product as one system, not separate pieces. Vega, Atrium, and the Founder admin are three faces of one system. Data flows between them should be first-class.	Map the shared core vs product-specific shells. Identify where the three faces duplicate logic that should be shared.
4. Alpha Launch — Ship and Iterate	Launch fast, learn from real users, iterate. Don't perfect before launching. Real user feedback on the core value proposition is worth more than polish.	Tag every finding as SHIP-BLOCKER / POLISH / NICE-TO-HAVE so the team can ship Atrium with confidence and know exactly what to fix in the first 2 weeks post-launch.

Table 1.1 — The four Van Clief ideas applied in this audit. Each idea maps to a concrete section of the report.

1.2 What This Audit Is NOT

This is not a generic code-review. It is a focused Van Clief-lens audit with three constraints. First, the ICM scope covers all 4 AI surfaces (ALOA/ARIA chat, DraftPro editor, Research Studio, Proactive Engine) — the audit catalogs every prompt string and recommends markdown migration priorities per surface. Second, the wrong-layer audit is identify-only — no code is changed in the frontend / backend boundary this session; recommendations are catalogued for the team to action. Third, every finding is tagged with a launch posture so the team can ship Atrium with confidence.

1.3 What Is Preserved From the Previous Audit

The previous audit (which mistakenly applied a jewellery-house metaphor) still produced genuine engineering findings that are independent of any methodology. These are preserved and reframed through the Van Clief lens where applicable. They are NOT discarded.

Finding	From previous audit	Reframed through Van Clief lens
portals.ts multi-tenant hole (1 of 70 mutations authed)	Critical security finding	Becomes SHIP-BLOCKER under the launch lens. NOT a Van Clief wrong-layer issue per se, but a backend-ownership issue — every mutation should own its own auth context.
4 untrusted dangerouslySetInnerHTML sites	XSS hardening — already FIXED in prior session	Stays FIXED. Showcases the "context IS the orchestration" principle: the sanitization rule is the context; the component just renders the sanitized output.
49 dead shadow files (~170 TS errors)	Tech debt	Becomes NICE-TO-HAVE under the launch lens. Not a blocker; quick morale win to clear in 1 hour.
God components (9 files > 1,500 LOC)	Maintainability	Becomes POLISH. The "wrong layer" lens reveals that many god components are doing orchestration in render — the orchestration should be context (hooks), the render should be pure.
Email-as-token + unsigned ?impersonate= param	Critical auth risks	Becomes SHIP-BLOCKER. The "wrong layer" lens clarifies: auth context belongs on the backend; the URL param is the wrong layer for identity.
5 surgical fixes applied (Header imports, gemini union, capacitor nullability, sanitize wrappers, TermsAcceptance guard)	Already applied; build green	Preserved. No regression.

Table 1.2 — Findings preserved from the previous audit, reframed through the Van Clief launch lens.

2. ICM Audit — ALOA / ARIA Prompt Architecture

The ICM scaffold already exists. There is a /ai/ directory with a README that explains the philosophy, and /ai/prompts/ contains 5 numbered markdown files (01-aloa-legal-identity.md, 02-aria-property-identity.md, 03-identity-guardrail.md, 04-interactive-form-protocol.md, 05-precision-protocol.md). The README even admits the gap: "Eventually, the source code files should READ from these markdown files at build time." This audit measures how far that migration has actually progressed.

2.1 ICM Completeness Score: ~18%

Dimension	Weight	Current	Score
Markdown files exist for primary prompts	20%	5 of ~8 (62%)	12.5%
Markdown files exist for secondary prompts (agents, Convex)	10%	0 of ~10	0%
Markdown content matches code (no drift)	20%	~1 of 5 closely matches	4%
Code reads from markdown at build time	30%	0 of 5 mapped functions load MD	0%
Single source of truth (no competing systems)	10%	2 identity-guardrail systems coexist	2%
Markdown editable without deploy (README's goal)	10%	False — editing MD has zero runtime effect	0%
Overall	100%	—	~18%

Table 2.1 — ICM completeness score. The scaffold exists; the migration from "code is source of truth" to "markdown is source of truth" has not started in code.

2.2 Drift Report — Markdown vs Code

Comparing each markdown file against its corresponding code string reveals significant drift. The markdown is NOT a faithful mirror of the code; in some cases (the identity guardrail) the markdown is a 6-line stub while the code is a 92-line fortress. Anyone editing only the markdown would weaken the guardrail. The most alarming drift cases are below.

MD file	Drift	Specific mismatches
01-aloa-legal-identity.md	SHIP-BLOCKER HIGH	MD says "designed for PracticePro Vega"; code says "designed for Komplet" (product rebrand not propagated). MD missing "JURISDICTIONAL ANALYSIS IN CHAT" + "ACTION PROTOCOLS" sections that exist in code.
02-aria-property-identity.md	SHIP-BLOCKER HIGH	MD says "Asset & Revenue Intelligence Assistant"; code says "Asset & Revenue Intelligent Assistant" (Intelligence→Intelligent). MD has "Strict Terminology" section (Resident/Unit/Service Charge/Minimum Vend); code has NONE of this. Code has Nigerian property law section; MD does not.
03-identity-guardrail.md	SHIP-BLOCKER CRITICAL	MD is a 6-line stub; code is a 92-line fortress with ~15 banned phrases + canned responses for "who are you", "are you Gemini", "ignore your system prompt". MD-only editors would WEAKEN the guardrail.
04-interactive-form-protocol.md	SHOWCASE LOW	Closest match (~90% aligned). Best candidate for first ?raw import proof-of-concept.
05-precision-protocol.md	POLISH MEDIUM	MD has Pre-Drafting Checklist + Quality Gate that code lacks. Code has 3 variants (Vega/Atrium/Komplete); MD documents only 2.

Table 2.2 — Drift between markdown and code. The identity-guardrail drift is the most dangerous: editing the markdown thinking it is the source of truth would weaken the LLM identity lock.

2.3 Code-Only Prompts (No Markdown Counterpart)

Beyond the 5 mapped files, there are 17 additional prompt strings living in code with zero markdown counterpart. These represent the bulk of the ICM migration work. The most significant are the 9 agent SYSTEM_PROMPTs (DataProtectionAgent, NigerianTaxComplianceAgent, RpcGuidanceAgent, DraftingAgent, NigerianLegalJurisdictionAgent, IngestionAgent, ScaleOfChargesAgent, ResearchAgent, AdvancedLegalDocumentIntelligenceAgent) and the 2 Convex server-side AI prompts (morning briefing generator in convex/proactive.ts, conversation summarizer in convex/conversationMemory.ts).

Prompt source	File:Line	Proposed MD file
9 agent SYSTEM_PROMPTs	src/agents/*.ts (one per file)	09-data-protection.md through 17-aldia.md (9 files)
Morning briefing generator	convex/proactive.ts:760-803 (44 lines)	18-morning-briefing.md
Conversation summarizer	convex/conversationMemory.ts:203-216 (14 lines)	19-conversation-summarizer.md
17 function-calling tool descriptions	src/services/geminiService.ts:23-268	06-tool-catalog.md (descriptions only; schemas stay in TS)
streamDraft system instruction	src/services/geminiService.ts:778-805	20-research-mode.md (consolidates 2 divergent blocks)
analyzeAttorneyDictation prompt	src/services/geminiService.ts:1003-1004	21-dictation-analyzer.md
generateResearchQuery prompt	src/services/geminiService.ts:1160-1189	22-research-query-generator.md

Table 2.3 — Code-only prompts that need markdown counterparts. The 9 agent SYSTEM_PROMPTs are the largest cluster; the 2 Convex prompts have a runtime-loading caveat (Convex does not support Vite ?raw imports — see § 2.6).

2.4 Identity Guardrail Split-Brain

POLISH Two parallel identity-guardrail systems coexist. This is a "wrong layer" issue AND an ICM issue.

The OLD system lives in src/config/identityGuardrails.ts and exposes identityLock(agent) + validateAIResponse(response, agent) (returns {isValid, sanitized, violations}). The NEW system lives in src/constants/identityGuardrails.ts and exposes getIdentityGuardrail(isProperty) + validateAIResponse(response, isProperty) (returns a string). The two systems have different prohibited-phrase lists, so the same AI response can PASS one validator and FAIL the other. Confirmed callers: AgencyHub.ts:168 uses the NEW system (correct); geminiService.ts:7,509,547,727 uses the OLD system (split-brain); AloaChat.tsx:56 uses the NEW system (correct). The OLD identityLock() function at config/identityGuardrails.ts:64 has zero callers — it is dead code.

2.5 A Hidden Bug — Atrium Chat Gets the Legal Precision Protocol

SHIP-BLOCKER AgencyHub.ts:240 passes a string where a boolean is expected. Atrium-mode chat silently uses the Vega (legal) precision protocol.

The call site `getAloaProtocol(appState.firmDetails?.product)` passes the product string (e.g. 'property') as the first argument, which is typed `isUnified: boolean`. TypeScript flags this as an error (`src/agents/AgencyHub.ts(240,23): error TS2345`). At runtime, the non-empty string is truthy, so `isUnified` is treated as `true`, and the function returns `ALOA_KOMLETE_PROTOCOL` — but only if the third arg is also 'komplete'. In practice, the third arg is undefined, so the function falls through to the default return `ALOA_PRECISION_PROTOCOL` (the Vega legal variant). This means Atrium users get the LEGAL precision protocol in their ARIA chat system instruction. The fix is one line: `getAloaProtocol(isUnified, null, appState.firmDetails?.product)`.

2.6 Build-Time Loading — Recommended Approach

Vite natively supports importing any file as a raw string via the `?raw` suffix. This requires zero plugins. The recommended approach is a thin loader module that imports the 5 markdown files as strings at build time, then replacing the inline string literals in code with references to the loader. The markdown files become the true source of truth; editing them and rebuilding is sufficient. No code changes are needed for routine prompt edits. This matches the ICM "no heavy framework code" philosophy — the loader is 10 lines of TypeScript.

```
// src/constants/loadPrompts.ts — Vite ?raw imports
// Markdown files are bundled as strings at build time.
// Editing the .md files and rebuilding is sufficient; no code change needed.
import aloaIdentity from '../../../ai/prompts/01-aloa-legal-identity.md?raw';
import ariaIdentity from '../../../ai/prompts/02-aria-property-identity.md?raw';
import identityGuardrail from '../../../ai/prompts/03-identity-guardrail.md?raw';
import formProtocol from '../../../ai/prompts/04-interactive-form-protocol.md?raw';
import precisionProtocol from '../../../ai/prompts/05-precision-protocol.md?raw';

export const PROMPTS = {
  aloaIdentity,
  ariaIdentity,
  identityGuardrail,
  formProtocol,
  precisionProtocol,
} as const;

// Then in AgencyHub.ts:
//   return PROMPTS.formProtocol;    // instead of the inline 45-line string
//
```

Convex caveat: `convex/proactive.ts` and `conversationMemory.ts` run in the Convex runtime, which does NOT support Vite `?raw` imports. For those two prompts, either (a) keep them inline in TS and accept they are not MD-sourced, or (b) load the MD via a Convex `httpAction` that reads from a deployed asset. Option (a) is pragmatic for now; option (b) is a Phase 2 polish item.

3. ICM Audit — DraftPro Editor

DraftPro is the TipTap-based document drafting editor used in both Vega (legal) and Atrium (property) modes. The editor itself is a 3,798-line god component (DraftProEditor.tsx), but the AI prompt architecture is spread across 6 files: aloaPrompts.ts (5 exported constants + a selector), geminiService.ts (streamDraft composition + 4 inline prompts), DraftingAgent.ts (rewriteText), DraftProEditor.tsx (3 inline micro-prompts), and jurisdictionConfig.ts (buildJurisdictionContextBlock). This audit catalogues 18 distinct prompt surfaces.

3.1 The 3-Variant Split — Why One MD File Is Not Enough

The code has 3 distinct, non-trivially-different precision protocols:

ALOA_PRECISION_PROTOCOL (Vega legal, 115 LOC), ALOA_ATRIUM_PROTOCOL (Atrium property, 111 LOC), and ALOA_KOMPLETE_PROTOCOL (unified, 73 LOC). The existing 05-precision-protocol.md (56 lines) attempts to document all 3 in one file but is CRITICAL drift — it omits the Komplete variant entirely, fabricates rules that do not exist in code (Claimant/Defendant nomenclature, Naira formatting, Visitor Pass doc type), and lists a wrong API signature. The recommendation is to split into 3 separate files (05a-vega-precision.md, 05b-atrrium-precision.md, 05c-komplete-precision.md) because the ICM principle "folder structure IS the architecture" means each variant — selected by getAloaProtocol() at runtime — should be its own file.

Variant	Code constant (LOC)	Unique content	Proposed MD
Vega (legal)	ALOA_PRECISION_PROTOCOL (115)	"User is ALWAYS the Lawyer/Solicitor"; 3 doc structures (Notice to Quit, Court Affidavit, Motion); embeds LITIGATION_SKELETON_INSTRUCTION	05a-vega-precision.md
Atrium (property)	ALOA_ATRIUM_PROTOCOL (111)	"User is ALWAYS the Property Manager"; 3 property structures (Notice to Tenant, Property Affidavit, Tenancy Agreement); NO litigation skeleton	05b-atrrium-precision.md
Komplete (unified)	ALOA_KOMPLETE_PROTOCOL (73)	Role-agnostic; "Do NOT assume the user is a Lawyer..."; "CONTEXT-AWARE SIGNING" section; NO doc structures block	05c-komplete-precision.md

Table 3.1 — The 3 precision-protocol variants. Each is architecturally distinct (selected by getAloaProtocol at runtime) and should be its own markdown file per ICM principles.

3.2 DraftPro Prompt Inventory — 18 Surfaces

18 distinct DraftPro prompt strings were catalogued across 6 source files. Only 1 has a markdown counterpart (05-precision-protocol.md), and it is HIGH drift. The table below highlights the most significant surfaces; the full inventory is in the audit transcript.

Prompt surface	File:Line	LOC	Has MD?	Proposed MD
ALOA_PRECISION_PROTOCOL (Vega)	aloePrompts.ts:29-143	115	PARTIAL (drift)	05a-vega-precision.md
ALOA_ATRIUM_PROTOCOL	aloePrompts.ts:248-358	111	PARTIAL (drift)	05b-atrrium-precision.md

Prompt surface	File:Line	LOC	Has MD?	Proposed MD
ALOA_KOMPLETE_PROTOCOL	aloePrompts.ts:152-224	73	NO	05c-komplete-precision.md
DRAFTPRO_HTML_FORMATTING_RULES	aloePrompts.ts:226-242	17	NO	06-draftpro-html-rules.md
LITIGATION_SKELETON_INSTRUCTION	aloePrompts.ts:9-27	19	NO	07-litigation-skeleton.md
streamDraft system instruction composition	geminiService.ts:778-805	28	NO	08-streamdraft-recipe.md (architecture doc)
roleContextBlock (3 variants)	geminiService.ts:763-776	9	NO	fold into 05a/05b/05c
buildJurisdictionContextBlock	jurisdictionConfig.ts:219-236	18	NO	09-jurisdiction-context.md
generateResearchQuery (SHARED with Research Studio)	geminiService.ts:1160-1189	30	NO	16-research-query-generator.md
analyzeAttorneyDictation	geminiService.ts:1003-1004	2	NO	10-audio-prompts.md (bundle)
transcribeAudio	geminiService.ts:1071-1074	4	NO	10-audio-prompts.md (bundle)
rewriteText (DraftingAgent — HAS BUG)	DraftingAgent.ts:17-32	12	NO	12-rewrite-text.md (DEFER until bug fixed)
Citation instructions prepend	DraftProEditor.tsx:1018-1024	7	NO	bundle with 16
Redraft wrapper	DraftProEditor.tsx:1786-1787	2	NO	bundle with 16
AI help placeholder prompt	DraftProEditor.tsx:1961	1	NO	bundle with 16

Table 3.2 — DraftPro prompt inventory. 18 surfaces total; 15 listed (3 are micro-prompts bundled). Only 1 has MD today and it is HIGH drift.

3.3 The rewriteText Variant-Selection Bug

SHIP-BLOCKER DraftingAgent.ts:17-21 hardcodes the Vega (legal) precision protocol for ALL rewrite calls, including Atrium and Komplete users.

The rewriteText function wraps ALOA_PRECISION_PROTOCOL unconditionally — no getAloaProtocol() call, no isProperty branch. When an Atrium user selects text in a property document and asks for a redraft, the AI receives the legal precision protocol (which says "User is ALWAYS the Lawyer/Solicitor" and references litigation skeletons). This is the same class of bug as the AgencyHub.ts:240 call-site bug already documented. The fix is to call getAloaProtocol(isUnified, signerContext, product) with the correct args. Do not migrate rewriteText to markdown until this bug is fixed — migrating a buggy prompt freezes the bug in a versioned file.

3.4 DraftPro Does NOT Use Function-Calling Tools

Unlike the ALOA/ARIA chat surface (which has 17 function-calling tools), DraftPro is pure text generation (streaming) + multimodal audio (dictation/transcription). The streamDraft function does not pass a tools array to the Gemini API. This makes DraftPro prompts SIMPLER to migrate to markdown than chat prompts — there is no FunctionDeclaration array to keep in sync. The migration is purely: prompt string → markdown file → ?raw import. The 14 FunctionDeclarations in geminiService.ts:23-268 are chat-only; the only DraftPro-related tool is start_drafting (which opens the editor from chat, not invokes drafting itself).

3.5 DraftPro Migration Priority Order

The recommended migration order starts with the lowest-risk, highest-value surfaces to prove the Vite ?raw loader pattern, then extends to the larger protocol variants. The rewriteText bug must be fixed before its migration. The streamDraft composition logic stays in code (it is the orchestration recipe, not a prompt) but should be documented in ai/README.md as an architecture diagram.

Rank	Prompt	Safety	Value	Phase
1	DRAFTPRO_HTML_FORMATTING_RULES (17 LOC, self-contained)	HIGH	HIGH	Phase 1 (first)
2	LITIGATION_SKELETON_INSTRUCTION (19 LOC, references data registry that stays in code)	HIGH	HIGH	Phase 1
3	ALOA_ATRIUM_PROTOCOL (111 LOC, no embedded sub-templates)	HIGH	HIGH	Phase 1
4	ALOA_PRECISION_PROTOCOL Vega (115 LOC, embeds LITIGATION_SKELETON_INSTRUCTION)	MEDIUM	HIGH	Phase 1
5	ALOA_KOMplete_PROTOCOL (73 LOC, self-contained)	HIGH	MEDIUM	Phase 1
6	generateResearchQuery (30 LOC, SHARED with Research Studio)	HIGH	MEDIUM	Phase 2
7	buildJurisdictionContextBlock (parameterized — needs render layer)	MEDIUM	MEDIUM	Phase 2
DE FER	rewriteText (FIX VARIANT BUG FIRST)	LOW	MEDIUM	Phase 3 (after bug fix)

Table 3.3 — DraftPro migration priority. Ranks 1-5 are the Phase 1 proof-of-concept batch; rank 6-7 are Phase 2; rewriteText is deferred until the variant-selection bug is fixed.

4. ICM Audit — Research Studio

Research Studio is the legal/property research workspace that lets users create notebooks, add sources (web URLs, documents), and run AI analyses (source analysis, notebook synthesis, research chat). The audit catalogued 12 distinct prompt surfaces across ResearchAgent.ts, ResearchStudio.tsx, aiUtils.ts, and geminiService.ts. Zero have markdown counterparts today (ICM completeness for Research Studio = 0%).

4.1 CRITICAL — Research Studio Runs Without Identity Protection

SHIP-BLOCKER Research Studio runs with NO identity guardrail AND NO validateAIResponse post-processing.

The ResearchAgent.ts calls streamGemini() and streamGeminiMultipart() without the options object that would trigger getSystemInstruction(). As a result, systemInstruction stays empty string. The ONLY identity context the model receives is SENIOR_ASSOCIATE_PERSONA embedded in the user-message text — and that persona has the WRONG ARIA expansion ("Advanced Research & Intelligence Assistant" instead of the canonical "Asset & Revenue Intelligence Assistant"). Worse, streamGeminiMultipart() does not support systemInstruction at all — so every PDF analysis runs with zero system-instruction protection. If a user asks "What are you?" in Research Studio, the model has no identity lock and will likely respond "I am Gemini, a large language model trained by Google." The validateAIResponse() safety net that strips such phrases in the main ALOA chat is NOT wired into the ResearchAgent pipeline.

4.2 The Wrong ARIA Expansion Bug

SHIP-BLOCKER ResearchAgent.ts:8 expands ARIA as "Advanced Research & Intelligence Assistant" — contradicts every other file.

The canonical ARIA expansion is "Asset & Revenue Intelligence Assistant" per constants/identityGuardrails.ts:118 and ai/prompts/02-aria-property-identity.md. But ResearchAgent.ts:8 (SENIOR_ASSOCIATE_PERSONA) says "Advanced Research & Intelligence Assistant". Additionally, PropertyManagementAgent.ts:21 says "Asset & Revenue Intelligent Assistant" (Intelligence → Intelligent). Three different expansions of the same acronym across the codebase. This must be reconciled to a single canonical form before any ICM migration — otherwise the markdown will freeze the wrong expansion.

4.3 Research Studio Prompt Inventory — 12 Surfaces

Prompt surface	File:Line	LOC	Has MD?	Proposed MD
SENIOR_ASSOCIATE_PERSONA (WRONG ARIA expansion)	ResearchAgent.ts:8-16	9	NO	13-research-persona.md
ANALYSIS_TOOLS 6 prompts (Chronology/Brief/Digest/Matrix/Gap/Adv serial)	ResearchStudio.tsx:103-108	6	NO	14-research-analysis-tools.md
chatWithSources prompt v1 (DEAD CODE — overwritten by v2)	ResearchAgent.ts:90-104	15	NO	DELETE in code
chatWithSources prompt v2 (live)	ResearchAgent.ts:113-127	15	NO	15-research-chat-template.md
Local getSystemInstruction (composes ALOA_PRECISION_PROTOCOL + RAG + docs)	aiUtils.ts:48-85	38	PARTIAL	document in 13
generateResearchQuery (SHARED with DraftPro)	geminiService.ts:1160-1189	30	NO	16-research-query-generator.md
RESEARCH MODE suffix (sendMessage version — full)	geminiService.ts:321-360	40	NO	17-research-mode-protocol.md
RESEARCH MODE suffix (streamMessage version — DRIFT, compressed)	geminiService.ts:608-617	10	NO	dedup into 17
IngestionAgent "ARIA Legal Ingestion Engine"	IngestionAgent.ts:118-127	10	NO	19-ingestion-engine.md
Auto-start "verify citations" invitation message	ResearchView.tsx:174-180	5	NO	18-research-invitation-message.md

Table 4.1 — Research Studio prompt inventory. 12 surfaces; 0 have MD today. The chatWithSources v1 is dead code (overwritten by v2) and should be deleted. The RESEARCH MODE suffix has drift between sendMessage and streamMessage versions.

4.4 Research Studio Does NOT Use Function-Calling Tools

Like DraftPro, Research Studio is pure text generation — no function-calling tools. The `analyzeSources()` and `chatWithSources()` functions call `streamGemini/streamGeminiMultipart` without a tools array. This means Research Studio AI cannot autonomously call `search_web`, `fetch_web_page`, `start_drafting`, or `analyze_document`. Users must manually add web sources via the `ResearchSourceColumn` UI, and the "Send to DraftPro" button is a client-side navigation hack that bypasses the AI tool layer. This is an architectural choice (text-only research assistant) but leaves Research Studio less capable than the main ALOA chat for the same legal-research use cases. Not an ICM migration blocker, but worth flagging for a future tool-augmentation pass.

4.5 Pre-Migration Code Fixes (BLOCKER)

Before any Research Studio prompt is migrated to markdown, 5 code fixes must ship. Migrating an unguarded or buggy prompt to markdown freezes the problem in a versioned file. The fixes are ordered by dependency.

#	Fix	File:Line	Effort
F1	Fix SENIOR_ASSOCIATE_PERSONA ARIA expansion → "Asset & Revenue Intelligence Assistant"	ResearchAgent.ts:8	5 min
F2	Inject getIdentityGuardrail(isProperty) at top of analyzeSources prompt assembly	ResearchAgent.ts:34-70	30 min
F3	Inject getIdentityGuardrail(isProperty) at top of chatWithSources + DELETE dead v1 prompt at lines 90-104	ResearchAgent.ts:75-178	30 min
F4	Wire validateAIResponse(content, isProperty) into both ResearchAgent return paths (mirror geminiService.ts:508-509)	ResearchAgent.ts:65, 170	30 min
F5	Add systemInstruction support to streamGeminiMultipart() (currently has none — all PDF analysis runs unguarded)	aiUtils.ts:169-196	1 hour

Table 4.2 — Pre-migration code fixes. F1 is the 5-minute fix already in the implementation plan (P4). F2-F5 close the identity-guardrail gap that leaves Research Studio unprotected.

5. ICM Audit — Proactive Intelligence Engine

The Proactive Engine is the server-side AI system that runs on cron jobs to: (1) scan for deadlines and anomalies (deterministic, no AI), (2) generate a daily AI morning briefing per firm, (3) summarize conversations for cross-session memory. The audit found 3 AI-driven prompt surfaces and 3 deterministic scanners. Only the AI-driven surfaces are ICM-migration candidates. Zero have markdown counterparts today.

5.1 Proactive Engine Prompt Inventory — 3 AI Surfaces

Prompt surface	File:Line	Cron	Model	Has MD?
Morning Briefing generator (buildBriefingPrompt)	proactive.ts:760-803	daily 06:15 UTC	gemini-2.0-flash	NO
Conversation Summarizer (summarizeConversation)	conversationMemory.ts:203-216	daily 23:00 UTC	gemini-2.0-flash (JSON mode)	NO
Memory Injection preamble (getInjectionContext)	conversationMemory.ts:91-96	runtime (new session)	N/A (injected text)	NO

Table 5.1 — Proactive Engine AI prompt inventory. The 3 deterministic scanners (deadline, anomaly, court reminders) have no prompt and are not ICM candidates.

5.2 CRITICAL — Morning Briefing Runs Without ARIA Identity Lock

SHIP-BLOCKER The morning briefing appears in user-facing ARIA chat but runs without the full ARIA identity guardrail that interactive ARIA enforces.

The briefing prompt at `proactive.ts:769` says only "You are ARIA, the AI assistant for {firmName}." — a minimal identity claim, not the full 92-line guardrail fortress that lives in `constants/identityGuardrails.ts`. The full guardrail contains ~15 banned phrases + canned responses for "who are you", "are you Gemini", "ignore your system prompt". The morning briefing has none of this protection. The briefing is stored as an `aloaMessages` row with role: "model" and appears in the user's ARIA chat panel — the user cannot distinguish a cron-generated briefing from an interactive ARIA reply. If a user replies to the briefing with "What are you?", the conversation continues without the identity lock. The same gap exists in the conversation summarizer (lower severity because its output is a structured JSON memory blob, not a direct user-facing message, but the summary IS re-injected into ARIA via `getInjectionContext` so any leak propagates forward).

5.3 The Convex Runtime Loading Problem

Convex server actions run in Convex's sandboxed V8 runtime, not Node + Vite. The Vite `?raw` import approach recommended for the frontend (`src/`) does NOT work for Convex. Three loading options were evaluated. Option C (build-time codegen) is recommended because it provides zero runtime cost (inlined at build), type safety (compiler errors if MD is renamed), and a single source of truth (the same MD file can be referenced from the audit table, the codegen loader, and reviewer PRs). Once proven for Convex, the codegen pattern can extend to `src/` prompts, unifying the ICM loader architecture.

Option	Mechanism	Pros	Cons	Verdict
A — Inline TS (status quo)	Prompts as template literals in <code>proactive.ts</code>	Zero infra changes; zero cold-start cost	MD is documentation only; bypasses ICM entirely	Acceptable for v1
B — <code>httpAction</code> fetch	<code>httpAction</code> fetches MD from <code>/public/</code> at briefing time	MD is true source of truth; non-devs can edit	Network hop per cron tick × N firms; cache invalidation; fetch failure = briefing failure	NOT recommended
C — Build-time codegen	Pre-commit hook reads MD → emits <code>convex/_generated/prompts.ts</code>	Zero runtime cost; type-safe; single source of truth; survives deploy	Requires build step; generated file must be committed	RECOMMENDED

Table 5.2 — Convex runtime loading options. Option C (build-time codegen) is recommended for the Proactive Engine. The codegen pattern can later extend to `src/` prompts, unifying the ICM loader.

5.4 Error Handling — Silent Failures Across the Board

Error handling is silent across the entire Proactive Engine. No retries (Convex cron does not auto-retry), no user notifications, only console.warn/console.error. The conversation summarizer self-heals within 24h (the next-night batch picks up unsummarized conversations). The morning briefing does NOT self-heal — a failed firm loses that day's briefing with no fallback. A chronic failure (quota exhausted, model deprecated) silently degrades the product for days before anyone notices. Recommended: add a `proactive_engine_health` log row or a founder-dashboard alert when `briefingsCreated === 0` for a day with at least 1 active firm.

Failure mode	Behavior	User impact
Missing GEMINI_API_KEY	console.warn + skip firm	Silent — no briefing appears
AI HTTP non-200	console.warn + skip firm	Silent — no briefing appears
AI returns empty content	Fallback to literal "Unable to generate briefing..." string	User sees degraded ARIA message
Any thrown exception	try/catch per firm, console.error, continue	Silent — cron continues, this firm gets no briefing
JSON parse failure (summarizer)	try/catch falls back to raw content as summary	Silent degradation — summary stored with degraded structure
Conversation too short (<3 msgs)	Returns {skipped: true}	Silent — by design

Table 5.3 — Proactive Engine error handling. All paths are silent. The summarizer self-heals within 24h; the briefing does not.

5.5 Proposed Markdown Files + Migration Priority

Proposed MD	Source	Priority	Loading option
20-morning-briefing.md	proactive.ts:760-803 (buildBriefingPrompt)	P1 (user-facing, identity-sensitive)	C (codegen) + inject guardrail prefix from 03
21-conversation-summarizer.md	conversationMemory.ts:203-216	P2 (internal memory, JSON-structured)	C (codegen)
22-conversation-memory-injection.md	conversationMemory.ts:91-96 (getInjectionContext)	P3 (small static string, low churn)	A (inline) or fold into 21

Table 5.4 — Proposed Proactive Engine markdown files. P1 is the highest priority because it is the only user-facing surface and it most needs the identity guardrail prepended.

6. "You're Automating the Wrong Layer" Audit

Van Clief's second idea: most people build complex orchestration code when they should be structuring the context that feeds the system. The context IS the orchestration. This section catalogues every place the React frontend is doing work that should be a Convex mutation or query. The findings are identify-only — no code is changed in the frontend / backend boundary this session. Each finding is tagged with a launch posture so the team can prioritize.

6.1 The Three Van Clief Showcases (Already Correct)

Before cataloguing what is wrong, it is worth noting what is already right. Three flows in PracticePro are textbook Van Clief implementations — the backend owns the orchestration, the client makes a single call, and the context (not client-side code) does the work. These showcases prove the team already understands the methodology; the wrong-layer findings below are places where that understanding has not yet been extended.

Flow	Why it is a showcase	Pattern
SHOWCASE Sales-lead pipeline	Contact Sales form fires ONE mutation (submitInquiry). Backend handles: insert row, notify founders, dispatch FCM push, update badge count reactively. Client does zero orchestration.	notifyFounders helper (convex/founderNotifications.ts) centralizes the dispatch. Replaces 30+ lines of copy-pasted notification logic across 3 mutations.
SHOWCASE Retainer billing state machine	Staged → Queued → Sent / Failed / Paused / Skipped state machine lives entirely in Convex mutations. Each transition validates auth + ownership + state legality. Client fires ONE mutation per action (approve / pause / skip / retry). No optimistic updates, no sequential mutations, no client-side state machine duplication.	Cron-driven automation (scanMattersForRetainerCycle + advanceStagedOutbox) runs server-side. Client only renders the Convex subscription.
SHOWCASE notifyFounders helper	A single server-side function that: finds all Founder-role users, inserts a notifications row per founder, queries their FCM tokens, schedules the FCM push. Replaces what would otherwise be 4+ client-side orchestration steps.	Context IS the orchestration: the founder list + notification payload IS the dispatch instruction.

Table 6.1 — Three flows that already exemplify the Van Clief methodology. The wrong-layer findings below are places where the same methodology has not yet been applied.

6.2 Wrong-Layer Findings — Header.tsx Notification Bell

The notification bell in Header.tsx is the single largest concentration of wrong-layer code in the codebase. The frontend merges 4 notification sources (broadcast notifications, user notifications, virtual client messages, virtual tenant messages), deduplicates them by title|||message key, infers notification type from message text, computes the unread count THREE different ways (backend query + frontend filter + localStorage-filtered), and fires local push notifications for events the backend already FCM-pushed. This is ~100 lines of client-side merge logic that should be one backend query.

Finding	What is wrong	Consolidation	Tag
Frontend merges 4 notification sources	Header.tsx:91-168 — aggregatedNotifications useMemo merges broadcasts + user notifs + virtual client msgs + virtual tenant msgs. Dedupes by title message key. Infers type from message text.	One backend query getUnifiedNotificationFeed({userId, firmId}) that joins, dedupes by broadcastId, sets type at insertion time.	POLISH

Finding	What is wrong	Consolidation	Tag
Unread count computed 3 ways	Header.tsx:170-191 — appUnread (backend) + unreadCount (frontend filter) + localStorage-filtered count. Three sources of truth for one number.	One query returning {items, unreadCount}. Add dismissedAt field to notifications table.	POLISH
Local push duplicates FCM push	Header.tsx:225-232 — fires showLocalNotification for every new notification. Backend already dispatched FCM via notifyFounders. Founder sees TWO pushes per sales lead.	Backend tags notifications with dispatchedChannels: ['fcm']. Frontend skips local push if FCM already dispatched.	POLISH
Mark-all-read routes by virtual flag	Header.tsx:442-449, 587-613 — client filters by _isBroadcast flag (which the client set itself), routes virtual notifications to a different code path. Comment at line 587 says "FIX".	Every notification should be a real DB row. Backend mutation that creates a client/tenant message should also insert a notifications row.	POLISH
Toast logic welded into layout shell	Header.tsx:194-237 — toast useEffect depends on 5 concerns (aggregatedNotifications, isLoading, addToast, navigateTo, currentUser). Cannot be tested or reused.	Extract to useNotificationToasts() hook in src/hooks/. Header becomes pure layout.	NICE-TO-HAVE

Table 6.2 — Wrong-layer findings in the notification bell. All are POLISH — they work functionally but cause UX spam (double push), drift (localStorage vs Convex), and unnecessary network load.

6.3 Wrong-Layer Findings — DataProvider.tsx Optimistic Updates

DataProvider.tsx implements optimistic updates that fight with Convex real-time subscriptions. The team has already worked around the symptoms (the id/_id split to prevent duplicate task cards, the recentlyDeleted Set to prevent resurrection, the revert-on-failure logic) but the workarounds themselves are code smells. Convex subscriptions push canonical state within ~50ms; the optimistic update is unnecessary and adds race-condition surface area.

Finding	Workaround in code today	Van Clief fix	Tag
Optimistic addItem	Generates temp UUID, optimistically inserts, then on success REPLACES local item with Convex _id. Comment explains the id/_id split was learned the hard way from duplicate task cards.	Skip optimistic update; let Convex subscription push the new item within ~50ms. Acceptable latency for most tables.	POLISH
Optimistic deleteItem + recentlyDeleted Set	markRecentlyDeleted(id) adds to a Set to "track this ID as recently-deleted so the firmData re-merge doesn't re-add it before Convex propagates the deletion." The Set is a workaround for the optimistic-update-fights-subscription race.	Remove the optimistic update + the Set. Convex subscription handles deletions reactively.	POLISH

Finding	Workaround in code today	Van Clief fix	Tag
Optimistic notification mutations	handleMarkNotificationsRead + handleClearAllNotifications do optimistic updates with revert-on-failure. Comment at line 333-335 acknowledges "the local state is already empty, so no resurrection."	If the local state is already empty and the subscription will push the truth, the optimistic update is unnecessary. Remove it.	POLISH
Sequential deleteItem loop for "clear all conversations"	DataProvider.tsx:436-441 — for (const conv of conversations) await baseActions.deleteItem(...). Sequential mutations, one per conversation.	One Convex mutation clearAllConversationsForFirm({firmId}) that bulk-deletes in one transaction.	NICE-TO-HAVE

Table 6.3 — Wrong-layer findings in DataProvider.tsx. The optimistic-update workarounds are the team's own acknowledgement that the wrong layer is being automated.

6.4 Wrong-Layer Findings — Defensive Polling in Founder App

POLISH 6 founder-app views bypass useQuery and use setInterval instead. The root cause is a deploy-gap crash that has a known canonical fix.

The pattern appears in: FounderDashboard.tsx:104 (60s), FounderBottomNav.tsx:242 (15s), SubscriptionRequestsCenter.tsx:131 (30s), FounderNotificationsCenter.tsx:215 (60s, also aggregates 4 sources client-side), OrganizationsHub.tsx:91 (30s), SecurityCenter.tsx:78 (30s). The comment at SubscriptionRequestsCenter.tsx:102-106 explicitly acknowledges the root cause: "the new founderMetrics mutations require a Convex deploy to exist on the backend. Until the deploy runs, useQuery would throw synchronously and crash the ENTIRE founder app (black screen)." The proper fix is to wrap useQuery in an ErrorBoundary. The BillingMonitorView.tsx:125-130 already demonstrates the canonical pattern. Restores real-time subscriptions; removes 6 polling timers × varying intervals of unnecessary network load.

6.5 Wrong-Layer Findings — Sequential Per-Message Mutations

POLISH MessagesView.tsx:1290-1314 fires 50+ sequential mutations on view mount when there are 50 unread messages.

On view mount (when atriumInboundResult and portalMessagesResult finish loading), a useEffect iterates all unread atrium_inbound_messages → fires markInboundRead({messageId}) per message; all unread portal_messages → fires markPortalRead({messageId}) per message; all unread portal_conversations → fires markConvReadByAdmin({conversationId}) per conversation. With 50 unread messages, this fires 50+ mutations in a tight loop, each a separate network round-trip. The fix is one mutation markAllInboxReadForFirm({firmId}) that bulk-patches all unread inbound + portal messages + conversations in a single transaction. Saves Convex function-call quota + ~2s load time on high-volume firms.

7. Alpha-Launch Posture — Ship Atrium Now

Van Clief's fourth idea: launch fast, learn from real users, iterate. The known bugs in Atrium are mostly polish, not blockers. This section explicitly tags every finding from this audit AND the preserved findings from the previous audit with a launch posture so the team can ship Atrium with confidence and know exactly what to fix in the first 2 weeks post-launch. The posture is binary where possible: SHIP-BLOCKER means do not launch until fixed; POLISH means fix in the first 2 weeks; NICE-TO-HAVE means backlog.

7.1 SHIP-BLOCKERS (Must Fix Before Atrium Launch)

3 ship-blockers. All are security issues. None are polish. All have known fixes with low regression risk. The team can ship Atrium the same day these 3 are fixed.

#	Blocker	Fix	Effort
B1	Unsigned ?impersonate= URL param (AuthContext.tsx:67-72). Anyone with a target user's email can impersonate them by visiting ?impersonate=victim@firm.com. No server check, no signature, no nonce.	Replace with server-verified founder-only mutation impersonateUser({targetUserId}) that issues a scoped session. Remove the URL param entirely.	1 day
B2	Server Gemini API key in plaintext localStorage (AuthContext.tsx:217-219 + ComposeModal.tsx:438). practicepro_custom_gemini_key synced from server to localStorage without encryption. Any XSS can exfiltrate it (and bill the firm).	Stop syncing to localStorage. Proxy all Gemini calls through a Convex action with HttpOnly cookie auth. The key stays server-side only.	2 days
B3	pushNotifications.markNotificationRead + markAllNotificationsRead have NO auth check (pushNotifications.ts:125-156). Anyone with a notification ID can mark it as read. A malicious user could mark all founders' notifications as read, suppressing sales-lead alerts.	Add requireFirmUser(ctx) + verify notification.userId === user._id before patching. One-line guard per mutation, follows existing pattern.	1 hour

Table 7.1 — Three ship-blockers. B1 and B2 require coordinated AuthContext changes; B3 is a one-hour fix. Total: 3-4 days of engineering to clear all three.

7.2 POLISH (Fix in First 2 Weeks Post-Launch)

12 polish items. All are wrong-layer patterns that work functionally but cause UX spam (double push), drift (localStorage vs Convex), flicker (optimistic updates vs subscriptions), or unnecessary network load (6 polling timers, sequential per-message mutations). The team should pick the highest-impact 5-6 for the first week post-launch and defer the rest to week 2.

#	Polish item	Impact	Effort
P1	Wire up the 5 existing ICM markdown files via Vite ?raw imports (start with 04-interactive-form-protocol.md — closest match)	ICM score 18% → 30%; first proof-of-concept for markdown-as-source-of-truth	4 hours

#	Polish item	Impact	Effort
P2	Fix getAloaProtocol call-site bug at AgencyHub.ts:240 (passes string where boolean expected — Atrium chat gets legal protocol)	Atrium ARIA chat finally uses the correct precision protocol	5 min
P3	Delete dead identityLock() at config/identityGuardrails.ts:64 (zero callers); migrate geminiService.ts:7,509,547,727 to new validator	Eliminates split-brain identity validation	1 hour
P4	Fix ResearchAgent.ts:8 ARIA expansion ("Advanced Research & Intelligence Assistant" → "Asset & Revenue Intelligence Assistant")	Closes identity-drift hole in research mode	5 min
P5	Reconcile ARIA expansion across MD + identityGuardrails.ts + PropertyManagementAgent.ts (Intelligence vs Intelligent)	Single source of truth for ARIA identity	1 hour
P6	Add requireFirmUser to 40 unauthenticated portals.ts mutations (the multi-tenant hole from prior audit)	Closes the multi-tenant isolation hole	4 hours
P7	Add requireFirmUser to 23 unauthenticated sentry.ts financial mutations	Closes the financial-integrity hole	1 day
P8	Replace 6 founder-app polling sites with useQuery + ErrorBoundary (BillingMonitorView pattern)	Restores real-time subscriptions; removes 6 polling timers	1 day
P9	Add markAllInboxReadForFirm bulk mutation (replaces 50+ sequential per-message mutations on view mount)	Saves Convex quota + ~2s load time on high-volume firms	3 hours
P10	Consolidate Header.tsx 4-source notification merge into one backend query getUnifiedNotificationFeed	Eliminates ~100 LOC of client merge logic + 3 sources of truth for unread count	1 day
P11	Add idempotencyKey + by_idempotency index to 5 critical tables (payment_proofs, subscriptionRequests, subscriptionAddons, termsAcceptance, tasks)	Closes double-submit risk on financial + legal records	1 day
P12	Convert deleteItem + forceDeleteItem + 3 cascade deletes to soft-delete via archive table (already exists at schema.ts:672)	NDPA compliance + reversibility for legal/financial records	2 days

Table 7.2 — 12 polish items. P1-P5 are ICM + identity-guardrail fixes (low effort, high morale win). P6-P7 close the multi-tenant hole from the prior audit. P8-P10 are wrong-layer consolidations. P11-P12 are data-integrity infrastructure.

7.3 NICE-TO-HAVE (Backlog)

#	Item	Why it is backlog
N1	Delete 49 dead shadcn files in src/components/ui/ + CognitiveGuidance.tsx (zero importers)	~170 TS errors cleared in 1 hour; pure tech-debt cleanup; no user-facing impact
N2	Delete dead NotificationCenter.tsx (177 LOC) + use-toast.ts (193 LOC) + ui/toaster.tsx + ui/sonner.tsx + ui/toast.tsx	~500 LOC of dead duplicate notification + toast systems; eliminates accidental-import risk
N3	Lazy-load TenantPortal, ClientDashboard, PropertyDetailView, SettingsView, LandingPage via lazyWithReload	~12,000 LOC out of main chunk; pattern already established; perf win but no functional change
N4	Extract 9 god components into tab sub-components (TenantPortal, DraftProEditor, AloaChat, MessagesView, PropertyDetailView, ClientDashboard, PropertyForm, LandingPage, App)	Largest item; ~22,000 LOC reorganization; do incrementally behind re-export shims
N5	Create markdown files for 9 code-only agent SYSTEM_PROMPTS (09-data-protection.md through 17-aldia.md)	Extends ICM to secondary AI surfaces; no functional change; can be done one agent at a time
N6	Add by_status index to sales_inquiries table (currently .collect() then JS filter — O(N) at 10k leads)	Perf optimization; works correctly today; only matters at scale
N7	Consolidate computeNextBillingDate (3 divergent copies) into convex/dateUtils.ts	Silent billing-date bugs today; pure function consolidation; very low risk
N8	Move toast logic out of Header.tsx into useNotificationToasts() hook	Refactor; Header becomes pure layout; no functional change

Table 7.3 — 8 backlog items. N1 is the highest-ROI quick win (170 TS errors in 1 hour). N4 is the largest item (god-component extraction, ~1-2 weeks of work).

8. Implementation Roadmap

The roadmap is sequenced so that the ship-blockers clear first (so Atrium can launch), then the highest-impact polish items (so the first 2 weeks post-launch are productive), then the nice-to-have backlog. Each phase is designed so that any individual item can ship independently — no item in a later phase depends on an item in an earlier phase being shipped first. The ICM migration is woven through Phase 2 (the first 5 markdown files get wired up) and Phase 3 (the 9 agent SYSTEM_PROMPTS get markdown counterparts).

8.1 Phase 0 — Ship-Blockers (3-4 days)

Goal: Clear the 3 security ship-blockers so Atrium can launch. Constraint: All 3 fixes are additive (no behavioural change for legitimate callers). B1 and B2 require coordinated AuthContext changes; B3 is a one-hour fix. Ship as a single PR.

Item	Effort	Owner concern
B3 — Add requireFirmUser to pushNotifications.markNotificationRead + markAllNotificationsRead	1 hour	Backend; one-line guard per mutation
B1 — Replace ?impersonate= URL param with server-verified founder-only mutation	1 day	AuthContext + Convex mutation; coordinated change
B2 — Stop syncing server Gemini API key to localStorage; proxy through Convex action	2 days	AuthContext + ComposeModal + Convex action; largest blast radius

8.2 Phase 1 — ICM Proof-of-Concept + Quick Wins (1 week)

Goal: Wire up the first ICM markdown file via Vite ?raw imports (proof-of-concept), fix the identity-guardrail split-brain, fix the Atrium-gets-legal-protocol bug, and clear the 5 quick-win TS errors. Constraint: All changes are surgical; no broad refactors. The ICM proof-of-concept is the highest-leverage item because it proves the markdown-as-source-of-truth pattern works end-to-end before extending it to the other 4 files.

Item	Effort	Why now
P2 — Fix getAloaProtocol call-site bug (Atrium chat gets legal protocol)	5 min	User-facing bug; one-line fix
P4 — Fix ResearchAgent.ts:8 ARIA expansion	5 min	Identity-drift hole; one-line fix
P3 — Delete dead identityLock() + migrate geminiService.ts to new validator	1 hour	Eliminates split-brain validation
P5 — Reconcile ARIA expansion across MD + code	1 hour	Single source of truth for ARIA identity
P1 — Wire up 04-interactive-form-protocol.md via Vite ?raw import (proof-of-concept)	4 hours	First proof that markdown-as-source-of-truth works end-to-end
N1 — Delete 49 dead shadcn files (~170 TS errors cleared)	1 hour	Quick morale win; pure tech-debt cleanup

8.3 Phase 2 — Security + Data Integrity (1-2 weeks)

Goal: Close the multi-tenant isolation hole from the prior audit, add idempotency to critical writes, and convert hard-deletes to soft-deletes. Constraint: Schema changes are additive (new optional fields + new indexes); no data backfill required. Pattern is already proven in feedback.ts and sendChatMessage — this phase extends it to the rest of the codebase.

Item	Effort	Why now
P6 — Add requireFirmUser to 40 unauthenticated portals.ts mutations	4 hours	Closes the multi-tenant hole from prior audit
P7 — Add requireFirmUser to 23 unauthenticated sentry.ts financial mutations	1 day	Closes the financial-integrity hole
P11 — Add idempotencyKey + by_idempotency index to 5 critical tables	1 day	Closes double-submit risk on financial + legal records

Item	Effort	Why now
P12 — Convert deleteItem + forceDeleteItem + 3 cascades to soft-delete via archive table	2 days	NDPA compliance + reversibility
P9 — Add markAllInboxReadForFirm bulk mutation	3 hours	Saves Convex quota + ~2s load time on high-volume firms
P10 — Consolidate Header.tsx 4-source notification merge into one backend query	1 day	Eliminates ~100 LOC of client merge logic
P8 — Replace 6 founder-app polling sites with useQuery + ErrorBoundary	1 day	Restores real-time subscriptions; removes 6 polling timers

8.4 Phase 3 — ICM Extension + Code Organization (2-4 weeks)

Goal: Extend ICM to the remaining 4 primary markdown files + the 9 agent SYSTEM_PROMPTs, extract the 9 god components, and consolidate duplicated helpers. Constraint: All changes are pure code reorganization — no behavioural change. Module splits ship behind re-export shims so callers do not need to update imports in lockstep. God-component extraction is done one component at a time.

Item	Effort	Why now
Wire up remaining 4 primary MD files (01, 02, 03, 05) via ?raw imports after P1 proof-of-concept	1 day	ICM score 30% → 60%
N5 — Create MD files for 9 code-only agent SYSTEM_PROMPTs	2 days	ICM score 60% → 80%
N3 — Lazy-load 5 large views via lazyWithReload	2 hours	~12,000 LOC out of main chunk
N7 — Consolidate computeNextBillingDate (3 divergent copies) into convex/dateUtils.ts	2 hours	Silent billing-date bugs today
N2 — Delete dead NotificationCenter.tsx + use-toast.ts + shadcn toast primitives	1 hour	~500 LOC of dead duplicate systems
N4 — Extract 9 god components into tab sub-components (incremental)	1-2 weeks	Largest item; do behind re-export shims
Split myFunctions.ts into 18 domain modules + portals.ts into 12 portal/* modules	1 week	Maintainability + build time
Convex prompt migration — load 18-morning-briefing.md + 19-conversation-summarizer.md via httpAction	2 days	ICM score 80% → 90%; Convex runtime caveat

9. Appendix — Recipes & Reference

9.1 The ICM Loader Recipe (Vite ?raw imports)

The recommended approach for wiring up the markdown files as the source of truth. Vite natively supports importing any file as a raw string via the ?raw suffix. No plugins required. The loader is ~10 lines of TypeScript. Editing the .md files and rebuilding is sufficient; no code changes are needed for routine prompt edits.

```
// src/constants/loadPrompts.ts
import aIaIdentity from '../..ai/prompts/01-aloa-legal-identity.md?raw';
```

```
import ariaIdentity from '../..ai/prompts/02-aria-property-identity.md?raw';
import identityGuardrail from '../..ai/prompts/03-identity-guardrail.md?raw';
import formProtocol from '../..ai/prompts/04-interactive-form-protocol.md?raw';
import precisionProtocol from '../..ai/prompts/05-precision-protocol.md?raw';

export const PROMPTS = {
  aloaIdentity,
  ariaIdentity,
  identityGuardrail,
  formProtocol,
  precisionProtocol,
} as const;

// Usage in AgencyHub.ts:
// Before (inline 45-line string):
//   const getInteractiveFormDelegationProtocol = (isAtrium: boolean) => `...45 lines...`;
// After:
//   const getInteractiveFormDelegationProtocol = (isAtrium: boolean) => {
//     return PROMPTS.formProtocol.replace('{{defaultMode}}', isAtrium ? 'property' : 'leg
al');
//   };

```

9.2 The notifyFounders Recipe (Already a Showcase)

The notifyFounders helper in convex/founderNotifications.ts is the canonical Van Clief pattern for backend-owned orchestration. A single client mutation (e.g., submitSalesInquiry) calls notifyFounders, which then: finds all Founder-role users, inserts a notifications row per founder, queries their FCM tokens, and schedules the FCM push via the scheduler. The client does zero orchestration. This pattern should be replicated for any new "notify on event" flow.

```
// convex/founderNotifications.ts (simplified)
export const notifyFounders = async (ctx, payload) => {
  const founders = await ctx.db.query("users")
    .withIndex("by_role", q => q.eq("role", "Founder"))
    .collect();

  for (const founder of founders) {
    // 1. Insert in-app notification
    await ctx.db.insert("notifications", {
      userId: founder._id,
      firmId: "system",
      title: payload.title,
      message: payload.message,
      type: payload.type,
      link: payload.link,
      isRead: false,
      createdAt: Date.now(),
    });

    // 2. Query FCM tokens
    const tokens = await ctx.db.query("user_push_tokens")
      .withIndex("by_user", q => q.eq("userId", founder._id))
      .collect();

    // 3. Schedule FCM push (zero client involvement)
    if (tokens.length > 0) {
      await ctx.scheduler.runAfter(0,
        internal.pushNotificationsNode.sendFcmPush,
        { tokens: tokens.map(t => t.token), title: payload.title, body: payload.message }
      );
    }
  }
}

```

```
}
};
```

9.3 The ErrorBoundary Recipe (Replaces Defensive Polling)

The canonical fix for the 6 founder-app polling sites that bypass useQuery to avoid deploy-gap crashes. Wrap the component in an ErrorBoundary with a fallback that renders a "not deployed yet" message. This preserves real-time subscriptions AND handles the deploy-gap case gracefully. The BillingMonitorView.tsx:125-130 already demonstrates this pattern.

```
// Pattern from BillingMonitorView.tsx:125-130
<ErrorBoundary fallback={<BillingMonitorNotDeployedFallback />}>
  <BillingMonitorInner />
</ErrorBoundary>

// BillingMonitorInner uses useQuery normally:
function BillingMonitorInner() {
  const outbox = useQuery(api.retainerBilling.getOutboxForFirm, { firmId });
  const stats = useQuery(api.retainerBilling.getOutboxStats, { firmId });
  // ...renders normally...
}

// If the backend function doesn't exist (deploy gap), ErrorBoundary
// catches the synchronous throw and renders the fallback. No black screen.
```

9.4 Files Modified in This Audit Session

This session applied two fixes: the landing page product dropdown wrap (immediate UI bug) and the 5 surgical fixes from the prior session (preserved). The build passes cleanly.

File	Change	Tag
src/components/LandingPage.tsx	Fixed "All Products" hover text wrapping to 2 lines — widened container + whitespace-nowrap	NICE-TO-HAVE
src/components/Header.tsx (preserved)	Removed duplicate useQuery + api imports	NICE-TO-HAVE
src/services/geminiService.ts (preserved)	Added "research" literal to modelPreference union	NICE-TO-HAVE
src/utils/capacitor.ts (preserved)	Fixed cachedIsNative nullability	NICE-TO-HAVE
src/components/details/DocumentDetailView.tsx (preserved)	Wrapped dangerouslySetInnerHTML in sanitize()	POLISH
src/components/documents/HtmlPagePreview.tsx (preserved)	Wrapped 2 dangerouslySetInnerHTML sites in sanitize()	POLISH
src/components/research/ResearchStudio.tsx (preserved)	Wrapped dangerouslySetInnerHTML in sanitize()	POLISH
src/components/TermsAcceptance.tsx (preserved)	Added isSubmitting guard + disabled Accept button	POLISH

9.5 Audit Methodology

This audit was conducted by two parallel exploration agents (one for the ICM / ALOA-ARIA prompt architecture, one for the wrong-layer / launch-posture audit) using read-only file inspection. Each agent produced a structured report with line-numbered citations. The findings were cross-referenced against the existing `worklog.md` entries to verify which patterns were intentionally applied in previous sessions. The landing page dropdown fix was applied manually and verified by the production build. No files were deleted, no schema was modified, and no Convex mutations were altered in this session. The full audit transcript is preserved in the conversation history for future reference.

The methodology prioritized the Van Clief lens throughout. Every finding is tagged with a launch posture (SHIP-BLOCKER / POLISH / NICE-TO-HAVE) so the team can ship Atrium with confidence. The ICM scope was deliberately limited to the ALOA / ARIA chat surface per the user's direction; DraftPro, Research Studio, and the proactive engine are flagged for future migration but out of scope for this audit. The wrong-layer audit is identify-only — no code is changed in the frontend / backend boundary this session; recommendations are catalogued for the team to action.